Les optionnels en Swift

420-249-AH - Programmation mobile iOS

1. Les optionnels

En Swift, un optional est une variable ou une constante qui peut contenir une valeur ou aucune valeur.

Les optionnels permettent à Swift de gérer explicitement les situations où une donnée peut être absente, ce qui contribue à rendre le code plus sûr et à réduire les erreurs d'exécution.

2. Pourquoi les optionnels existent-ils ?

Dans une application mobile, une donnée n'est pas toujours disponible.

Exemples :

- Une personne n'a pas encore ajouté de photo de profil
- La position GPS n'est pas encore disponible
- Un champ de formulaire est vide
- Une requête réseau n'a pas encore retourné de résultat
- Une préférence n'a jamais été enregistrée
- Une valeur recherchée n'existe pas

Dans ces situations, il faut pouvoir représenter une valeur ou l'absence de valeur. Swift utilise pour cela les optionnels.

3. Une variable normale doit contenir une valeur

Exemple :

```swift
var nom: String = "Alex"
```

nom doit toujours contenir un String. Ceci n'est pas permis :

```swift
nom = nil
```

Pourquoi ? Parce que String signifie : cette variable contient obligatoirement une chaîne de caractères.

4. La valeur nil

nil représente l'absence de valeur.

Exemple conceptuel :

```
courriel
   ↓
aucune valeur disponible
```

En Swift :

```swift
var courriel: String? = nil
```

Le symbole ? indique que la variable est optionnelle.

5. Type normal et type optionnel

Comparons :

```
var nom: String = "Alex"
```

et :

```
var nom: String? = "Alex"
```

Déclaration	Signification
`String`	Contient obligatoirement un texte
`String?`	Peut contenir un texte OU nil

6. Déclarer un optional

Syntaxe :

```
var nom: String?
```

Cette variable peut contenir :

```
nom = "Alex"
```

ou :

```
nom = nil
```

Autres exemples :

```
var age: Int?
var temperature: Double?
var estConnecte: Bool?
```

7. Exemple - Photo de profil

Dans une application mobile, une photo de profil peut être disponible ou non.

```
var photoProfil: String?
```

Au départ :

```
photoProfil = nil
```

Plus tard :

```
photoProfil = "profil.jpg"
```

On peut représenter les deux états avec la même variable.

8. Exemple - Position GPS

Une position GPS peut prendre quelques secondes avant d'être disponible.

```
var latitude: Double?
var longitude: Double?
```

Au démarrage :

```
latitude = nil
longitude = nil
```

Lorsque la position est obtenue :

```
latitude = 45.5019
longitude = -73.5674
```

Les optionnels sont donc particulièrement utiles lorsqu'une donnée peut arriver plus tard.

9. Un optional n'est pas exactement sa valeur

Considérons :

```
var age: Int? = 25
```

age n'est pas directement un Int. Son type est :

```
Int?
```

On peut visualiser cela comme une boîte :

```
┌──────────────┐
│ Optional<Int>│
│              │
│      25      │
└──────────────┘
```

ou :

```
┌──────────────┐
│ Optional<Int>│
│              │
│      nil     │
└──────────────┘
```

Il faut donc extraire la valeur avant de l'utiliser dans plusieurs situations.

10. Pourquoi Swift impose cette vérification ?

Considérons :

```
var age: Int? = nil
```

Puis imaginons :

```
let prochainAge = age + 1
```

Que devrait faire le programme ?

```
nil + 1 = ?
```

Cette opération n'a pas de sens. Swift oblige donc à vérifier qu'une valeur existe avant de l'utiliser. Cette approche permet d'éviter de nombreuses erreurs d'exécution.

11. Vérifier un optional avec nil

On peut tester directement :

```
var courriel: String? = nil

if courriel == nil {
    print("Aucun courriel disponible")
}
```

Ou :

```
if courriel != nil {
    print("Un courriel est disponible")
}
```

Mais savoir qu'une valeur existe ne suffit pas toujours. Il faut souvent récupérer cette valeur.

12. Déballer un optional

Récupérer la valeur contenue dans un optional est appelé Optional Unwrapping, ou déballage d'un optionnel.

Swift propose plusieurs techniques :

```
if let
guard let
??
optional chaining (?.)
!
```

Elles ne présentent pas toutes le même niveau de sécurité.

13. Optional Binding avec if let

if let permet de vérifier qu'un optional contient une valeur et de récupérer cette valeur.

Exemple :

```swift
var courriel: String? = "alex@email.com"

if let courrielUtilisateur = courriel {
    print(courrielUtilisateur)
}
```

Résultat :

```
Console
alex@email.com
```

14. Comprendre if let

Avec :

```swift
if let courrielUtilisateur = courriel {
    print(courrielUtilisateur)
}
```

Swift effectue conceptuellement :

```
courriel contient-il une valeur ?
          │
     ┌────┴─────┐
   Oui         Non
    │           │
    ▼           ▼
Extraire    Ne pas
la valeur   exécuter
    │
    ▼
courrielUtilisateur
```

15. Ajouter un else

```swift
var courriel: String? = nil

if let courrielUtilisateur = courriel {
    print("Courriel : \(courrielUtilisateur)")
} else {
    print("Aucun courriel disponible")
}
```

Résultat :

```
Console
Aucun courriel disponible
```

16. Exemple - Nom d'utilisateur

```swift
var nomUtilisateur: String? = "Alex"

if let nom = nomUtilisateur {
    print("Bienvenue \(nom)")
} else {
    print("Bienvenue")
}
```

Le programme fonctionne dans les deux situations : nom disponible ou nom absent.

17. Syntaxe simplifiée de if let

Lorsque le nouveau nom doit être identique à l'optional, Swift permet une syntaxe plus courte.

Au lieu de :

```swift
if let courriel = courriel {
    print(courriel)
}
```

on peut écrire :

```swift
if let courriel {
    print(courriel)
}
```

Cette syntaxe signifie : si courriel contient une valeur, crée une valeur locale non optionnelle portant le même nom.

18. Plusieurs optionnels avec if let

On peut vérifier plusieurs optionnels en même temps.

```swift
var latitude: Double? = 45.5019
var longitude: Double? = -73.5674

if let latitude, let longitude {
    print("Latitude : \(latitude)")
    print("Longitude : \(longitude)")
}
```

Le bloc est exécuté uniquement si les deux valeurs sont disponibles.

19. Ajouter une condition à if let

Il est également possible de combiner le déballage avec une condition.

```swift
var batterie: Int? = 15

if let batterie, batterie <= 20 {
    print("Attention : batterie faible")
}
```

Ici : la batterie doit exister ET être inférieure ou égale à 20.

20. guard let

guard let permet également de déballer un optional. Il est particulièrement utile dans une fonction lorsqu'une valeur est indispensable pour continuer.

Exemple :

```swift
func afficherProfil(nom: String?) {
    guard let nom else {
        print("Nom non disponible")
        return
    }
    print("Profil de \(nom)")
}
```

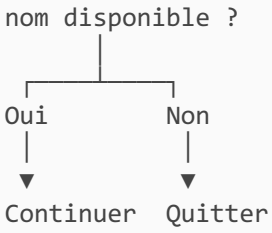
21. Comprendre guard let

Avec :

```swift
guard let nom else {
    return
}
```

on peut lire : vérifier que nom contient une valeur. Sinon, quitter immédiatement.

Conceptuellement :



22. if let ou guard let ?

if let - pratique lorsqu'on veut traiter les deux possibilités :

```
if let courriel {
    print(courriel)
} else {
    print("Aucun courriel")
}
```

guard let - pratique lorsqu'une valeur est obligatoire pour continuer :

```
guard let courriel else {
    return
}
print(courriel)
```

On élimine rapidement le cas invalide.

23. Exemple - Validation d'une donnée

```
func envoyerNotification(message: String?) {
    guard let message else {
        print("Impossible d'envoyer une notification vide")
        return
    }
    print("Notification envoyée : \(message)")
}
```

Appel :

```
envoyerNotification(message: "Nouvelle mise à jour")
```

Résultat :

```
Console
Notification envoyée : Nouvelle mise à jour
```

24. Valeur par défaut avec ??

Swift propose l'opérateur ??, appelé nil-coalescing operator.

Il permet d'utiliser une valeur par défaut lorsqu'un optional contient nil.

25. Exemple avec ??

```
var nom: String? = nil
let nomAffiche = nom ?? "Utilisateur"
print(nomAffiche)
```

Résultat :

```
Console
Utilisateur
```

Si :

```
nom = "Alex"
```

alors le même code produira :

```
Console
Alex
```

26. Comprendre ??

```
let resultat = optional ?? valeurParDefaut
```

Signifie :

```
optional contient une valeur ?
          |
     ┌────┴────┐
    Oui       Non
     |         |
     ▼         ▼
utiliser  utiliser
la valeur  la valeur
           par défaut
```

27. Exemple - Préférence utilisateur

Supposons qu'une préférence de volume puisse être absente.

```
var volumeSauvegarde: Int? = nil
```

On souhaite utiliser 50 par défaut.

```
let volume = volumeSauvegarde ?? 50
```

Résultat :

```
Console
50
```

Si une valeur existe :

```
volumeSauvegarde = 80
```

alors le même code donnera :

```
Console
80
```

28. Exemple - Nom affiché

```swift
var prenom: String? = nil
print("Bonjour \(prenom ?? "Utilisateur")")
```

Résultat :

```
Console
Bonjour Utilisateur
```

Avec :

```swift
prenom = "Alex"
```

Résultat :

```
Console
Bonjour Alex
```

29. Optional Chaining

Swift permet d'appeler une propriété ou une méthode sur un optional sans le déballer manuellement. On utilise ?.

Exemple :

```swift
var nom: String? = "Alex"
let longueur = nom?.count
```

Si nom contient "Alex", alors longueur contient :

```
Console
4
```

30. Que se passe-t-il si la valeur est nil ?

```swift
var nom: String? = nil
let longueur = nom?.count
```

Le programme ne plante pas. longueur devient simplement nil. Son type est :

```swift
Int?
```

31. Exemple avec une méthode

```swift
var ville: String? = "Montréal"
let villeMajuscule = ville?.uppercased()
```

Résultat :

```
Console
MONTRÉAL
```

Si :

```
ville = nil
```

alors ville?.uppercased() retourne :

```
Console
nil
```

32. Combiner ?. et ??

Ces deux opérateurs fonctionnent très bien ensemble.

```
var nom: String? = nil
let longueur = nom?.count ?? 0
print(longueur)
```

Résultat :

```
Console
0
```

Si :

```
nom = "Alex"
```

Résultat :

```
Console
4
```

33. Exemple - Texte utilisateur

```
var commentaire: String? = "Excellent service"
let nombreCaracteres = commentaire?.count ?? 0
print("Nombre de caractères : \(nombreCaracteres)")
```

Si aucun commentaire n'existe :

```
commentaire = nil
```

le résultat devient :

```
Console
Nombre de caractères : 0
```

34. Le déballage forcé avec !

Swift permet de forcer l'accès à la valeur d'un optional avec !.

Exemple :

```swift
var nom: String? = "Alex"
print(nom!)
```

Résultat :

```
Console
Alex
```

Le symbole ! signifie essentiellement : je garantis qu'une valeur existe.

35. Le danger de !

Considérons :

```swift
var nom: String? = nil
print(nom!)
```

Le programme essaie d'extraire une valeur qui n'existe pas.

Résultat : erreur d'exécution - l'application peut s'arrêter brutalement.

36. Pourquoi éviter ! ?

Avec nom!, on demande à Swift de ne plus nous protéger. On affirme : je suis certain qu'une valeur existe.

Si cette affirmation est fausse : crash.

Il est donc préférable, dans la majorité des situations, d'utiliser :

```swift
if let
guard let
??
?.
```

37. Comparaison des approches

Supposons :

```swift
var nom: String? = nil
```

Approche	Niveau de sécurité
`nom!`	Risqué
`if let nom { ... }`	Sécuritaire
`nom ?? "Utilisateur"`	Sécuritaire + valeur par défaut
`nom?.count`	Sécuritaire + opération seulement si la valeur existe

38. Exemple - Authentification

Supposons qu'un identifiant utilisateur puisse être absent.

```
var identifiantUtilisateur: Int? = 125
```

On peut écrire :

```
if let identifiant = identifiantUtilisateur {
    print("Utilisateur connecté : \(identifiant)")
} else {
    print("Aucun utilisateur connecté")
}
```

On évite :

```
print(identifiantUtilisateur!)
```

car rien ne garantit nécessairement qu'une session existe.

39. Exemple - Localisation

```
var latitude: Double? = 45.5019
var longitude: Double? = nil
```

On ne devrait utiliser la position que lorsque les deux valeurs sont disponibles.

```
if let latitude, let longitude {
    print("Position disponible")
    print("Latitude : \(latitude)")
    print("Longitude : \(longitude)")
} else {
    print("Position GPS non disponible")
}
```

40. Exemple - Formulaire

Supposons :

```
var nom: String? = "Alex"
var courriel: String? = nil
```

Une inscription nécessite les deux données.

```
if let nom, let courriel {
    print("Inscription de \(nom)")
    print("Courriel : \(courriel)")
} else {
    print("Informations manquantes")
}
```

41. Les optionnels avec les conditions

Les optionnels peuvent être combinés avec les expressions booléennes vues précédemment.

```
var batterie: Int? = 18
var modeEconomie = true

if let batterie,
   batterie <= 20 &&
   modeEconomie {
    print("Mode économie recommandé")
}
```

On combine ici : Optional Binding + comparaison + Bool + opérateur &&.

42. Les optionnels avec les fonctions

Une fonction peut recevoir un optional :

```
func afficherNom(_ nom: String?) {
    if let nom {
        print(nom)
    } else {
        print("Nom inconnu")
    }
}
```

Une fonction peut également retourner un optional :

```
func obtenirCode(estConnecte: Bool) -> Int? {
    if estConnecte {
        return 1234
    }
    return nil
}
```

Utilisation :

```
let code = obtenirCode(estConnecte: true)
if let code {
    print("Code : \(code)")
}
```

43. Pourquoi retourner un optional ?

Une fonction peut ne pas réussir à produire une valeur.

Exemples mobiles :

- Chercher une donnée
- Obtenir une position

- Charger une préférence
- Convertir une saisie
- Retrouver un utilisateur
- Lire une configuration

Situation	Comportement
`Valeur trouvée`	Retourner la valeur
`Valeur absente`	Retourner nil

44. Conversion et optionnels

Certaines conversions Swift retournent naturellement un optional.

Exemple :

```
let texte = "25"
let age = Int(texte)
```

Le type de age est :

```
Int?
```

Pourquoi ? Parce que cette conversion peut échouer.

45. Exemple de conversion réussie

```
let texte = "25"
let age = Int(texte)

if let age {
    print("Âge : \(age)")
}
```

Résultat :

```
Console
Âge : 25
```

46. Exemple de conversion impossible

```
let texte = "bonjour"
let age = Int(texte)
```

Swift ne peut pas convertir "bonjour" en Int. Donc age contient :

```
nil
```

et non une erreur d'exécution.

47. Exemple - Validation d'une saisie numérique

Une zone de saisie fournit souvent du texte.

Supposons :

```
let saisie = "85"
```

On souhaite obtenir un niveau de batterie :

```
let niveau = Int(saisie)
```

Puis :

```
if let niveau {
    if niveau >= 0 && niveau <= 100 {
        print("Niveau valide : \(niveau)%")
    } else {
        print("Valeur hors limites")
    }
} else {
    print("Veuillez saisir un nombre")
}
```

Ce scénario montre très bien l'intérêt des optionnels.

48. if let avec une conversion

On peut combiner directement la conversion et le déballage.

```
let saisie = "85"
if let niveau = Int(saisie) {
    print("Niveau : \(niveau)%")
} else {
    print("Valeur invalide")
}
```

C'est une construction très fréquente en Swift.

49. Exemple - Configuration d'une application mobile

Données :

```
var nomUtilisateur: String? = "Alex"
var niveauBatterie: Int? = 15
var emplacement: String? = nil
```

Afficher le nom de la personne. Si aucun nom n'existe : Utilisateur. Utiliser ??.

Pour la batterie : si une valeur existe et qu'elle est ≤ 20, afficher une alerte. Utiliser if let.

Pour l'emplacement : si une valeur existe, afficher l'emplacement, sinon afficher "Localisation indisponible".

50. Exercice - Profil utilisateur incomplet

On souhaite gérer un profil dont certaines informations peuvent être absentes.

Déclarer :

```
var nom: String?
var courriel: String?
var age: Int?
var ville: String?
```

Utiliser par exemple :

Donnée	Valeur
Nom	Alex
Courriel	nil
Âge	24
Ville	Montréal

Travail demandé :

1. Nom

Afficher :

```
Console
Utilisateur : Alex
```

Si le nom est absent :

```
Console
Utilisateur : Inconnu
```

Utiliser :

```
??
```

2. Courriel

Si le courriel existe :

```
Console
Courriel : ...
```

Sinon :

```
Console
Aucun courriel enregistré
```

Utiliser :

```
if let
```

3. Âge

Si l'âge existe et qu'il est supérieur ou égal à 18 :

```
Console
Compte adulte
```

Sinon, afficher un message approprié.

4. Ville

Afficher le nombre de caractères du nom de la ville. Si aucune ville n'existe :

```
Console
Nombre de caractères : 0
```

Utiliser :

```
?.
??
```

51. Exercice - Validation d'une saisie

Créer :

```
let saisieBatterie = "75"
```

Convertir cette valeur en Int. La conversion doit être traitée de façon sécuritaire.

Si la conversion réussit et que la valeur est comprise entre 0 et 100, afficher :

```
Console
Batterie : 75 %
```

Sinon :

```
Console
Niveau de batterie invalide
```

Tester ensuite avec :

```
"abc"
"-10"
"150"
"20"
```

52. Exercice - Coordonnées GPS

Créer :

```
var latitude: Double?
var longitude: Double?
```

Afficher :

```
Console
Position GPS disponible
Latitude : ...
Longitude : ...
```

uniquement lorsque les deux valeurs existent.

Sinon :

```
Console
En attente de la position GPS...
```

Tester plusieurs situations :

- latitude présente / longitude présente
- latitude présente / longitude nil
- latitude nil / longitude présente
- latitude nil / longitude nil

53. Quel outil choisir ?

Besoin	Outil
J'ai besoin d'une valeur seulement si elle existe	if let
La valeur est obligatoire pour poursuivre une fonction	guard let
Je veux une valeur par défaut	??
Je veux effectuer une opération seulement si la valeur existe	?.
Je suis absolument certain que la valeur existe	!

Attention : utiliser ! avec beaucoup de prudence.

54. À retenir

Un optional représente une valeur OU nil.

Syntaxe :

```
String?
Int?
Double?
Bool?
```

Pour récupérer une valeur de façon sécuritaire :

```
if let valeur = optional {
}
```

ou :

```
if let optional {
}
```

Lorsqu'une valeur est indispensable dans une fonction :

```
guard let valeur = optional else {
    return
}
```

Pour fournir une valeur par défaut :

```
optional ?? valeurParDefaut
```

Pour accéder de façon sécuritaire à une propriété ou une méthode :

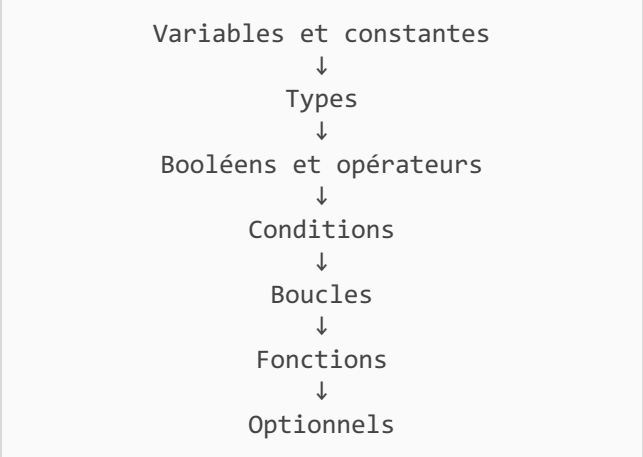
```
optional?.propriete
optional?.methode()
```

Le déballage forcé optional! doit être utilisé avec prudence, car :

Règle : optional == nil + ! = erreur d'exécution.

55. Vision globale

Les notions vues jusqu'ici s'enchaînent maintenant ainsi :



Les optionnels introduisent une idée fondamentale de Swift :

> Le langage oblige à prendre en compte explicitement l'absence possible d'une valeur.

Cette sécurité sera importante dans pratiquement tous les types d'applications mobiles : saisie utilisateur, données provenant de l'appareil, fichiers, configuration, localisation et services externes.